作业二

左之睿 191300087 人工智能学院 人工智能学院选修 本科

1、第三章习题1

(a)编译成功截图如下：

```
MEX completed successfully.
            MEX toolbox/mex/mexa64/vl_slic.mexa64
Building with 'gcc'.
In file included from /home/auror/Downloads/vlfeat-0.9.21/./toolbox/slic/vl_slic.c:14:0:
toolbox/mexutils.h: In function ‘vlmxError’:
toolbox/mexutils.h:277:1: warning: ‘noreturn’ function does return
 }
 ^

MEX completed successfully.
            MEX toolbox/mex/mexa64/vl_gmm.mexa64
Building with 'gcc'.
In file included from /home/auror/Downloads/vlfeat-0.9.21/./toolbox/gmm/vl_gmm.c:15:0:
toolbox/mexutils.h: In function ‘vlmxError’:
toolbox/mexutils.h:277:1: warning: ‘noreturn’ function does return
 }
 ^

MEX completed successfully.
            MEX toolbox/mex/mexa64/vl_aibhist.mexa64
Building with 'gcc'.
MEX completed successfully.
            MEX toolbox/mex/mexa64/vl_aib.mexa64
Building with 'gcc'.
In file included from /home/auror/Downloads/vlfeat-0.9.21/./toolbox/aib/vl_aib.c:16:0:
toolbox/mexutils.h: In function ‘vlmxError’:
toolbox/mexutils.h:277:1: warning: ‘noreturn’ function does return
 }
 ^

MEX completed successfully.
            MEX toolbox/mex/mexa64/vl_erfill.mexa64
Building with 'gcc'.
MEX completed successfully.
            MEX toolbox/mex/mexa64/vl_mser.mexa64
Building with 'gcc'.
In file included from /home/auror/Downloads/vlfeat-0.9.21/./toolbox/mser/vl_mser.c:14:0:
toolbox/mexutils.h: In function ‘vlmxError’:
toolbox/mexutils.h:277:1: warning: ‘noreturn’ function does return
 }
 ^

MEX completed successfully.
             MK toolbox/noprefix/
auror@auror-virtual-machine:~/Downloads/vlfeat-0.9.21$
```

Matlab中成功输出版本如下：

```
>> run("/home/auror/Downloads/vlfeat-0.9.21/toolbox/vl_setup.m")
>> vl_version
0.9.21
>>
```

(b)matlab代码如下图：

```
T3_1.m
N=5000;
x=rand(5000,10);
NN_idx=[];% 每个记录最近样本的idx
NN_dis=[];
% 开始计算最近邻；
t1=cputime;
for i = 1:N
    min_dis=10000;
    min_idx=10000;
    for j = 1:N
        if j~=i
            temp_dis=sqrt((x(0*N+i)-x(0*N+j))^2+(x(1*N+i)-x(1*N+j))^2+(x(2*N+i)-x(2*N+j))^2+(x(3*N+i)-x(3*N+j))^2+(x(4*N+i)-x
            temp_idx=j;
            if temp_dis<min_dis
                min_dis=temp_dis;
                min_idx=temp_idx;
            end
        end
    end
    NN_idx(i)=min_idx; %第i个元素对应的最近邻索引
    NN_dis(i)=min_dis;
end
t2=cputime;
t=t2-t1;
```

运行结果如下

首先是最近邻的距离和索引：

NN_dis

1x5000 double

	1	2	3	4	5	6	7	8	9	10	11	12	13
1	0.4118	0.3777	0.3334	0.4202	0.3756	0.4621	0.2749	0.4757	0.3056	0.5020	0.5071	0.4315	0.3694
2													
3													
4													

NN_dis NN_idx

1x5000 double

	1	2	3	4	5	6	7	8	9	10	11	12	13
1	4235	3702	3060	1147	2131	1053	785	3734	4565	4075	3902	3810	1754
2													

接下来是运行时间：

```
>> t

t =

    1.9062
```

(c)代码如下：

Editor - /home/auror/Desktop/vlfeat.m *

vlfeat.m *

```
run("/home/auror/Downloads/vlfeat-0.9.21/toolbox/vl_setup.m");
x=rand(5000,10); %5000*10
NN_idx=[];
NN_dis=[];
kdtree=vl_kdtreebuild(x','NumTrees',1);
t1=cputime;
for i =1:5000
    Q=x(i,:);
    [index, distance] = vl_kdtreequery(kdtree, x', Q', 'NumNeighbors', 2, 'MaxComparisons', 500) ;
    NN_idx(i)=index(2,1);
    NN_dis(i)=distance(2,1);
end
t2=cputime;
t=t2-t1
```

Command Window

由于vl_kdtreequery找的是最近邻，因此需要将NumNeighbors设置为2，并且取第二个值作为索引和距离，否则找到的最近邻就是这个点本身

找到的索引和距离如下：

Editor - vlfeat.m　　Variables - NN_idx

NN_idx

1x5000 double

	1	2	3	4	5	6	7	8	9	10	11
1	4787	4293	815	213	885	1482	4707	4269	1359	2408	1384
2											
3											

Editor - vlfeat.m　　Variables - NN_dis

NN_idx　NN_dis

1x5000 double

	1	2	3	4	5	6	7	8	9	10	11
1	0.1603	0.1902	0.1157	0.2155	0.1451	0.2104	0.2515	0.1538	0.1421	0.2127	0.1263
2											
3											
4											

耗时如图

```
>> t

t =

    7.8000
```

(d)用vlfeat找到的最近邻和按照准确计算距离找的最近邻作比较从而计算错误率。

实验中错误率很低，接近于0，下图是一次实验中的错误率：

Command Window

New to MATLAB? See resources for Getting Started.

```
t =

    5.2400


err =

     0

>>
```

(e)Numtrees增大时，运行时间变长，错误率没有明显变化；

MaxNumComparisons增大或减小对运行时间没有明显影响，其值越大错误率越低，且大到一定程度后错误率为0

(f)数据集变小时，运行时间减小，但只要MaxNumComparisons足够大，错误率就为0，若MaxNumComparisons较小(远小于数据集的规模时)，则数据集越小错误率越低；
数据集变大时，运行时间增大，错误率同样与MaxNumComparisons挂钩，当MaxNumComparisons小于数据集规模时，数据集越大，错误率越高。

2、第四章习题5

(a)表格补全如下：

下标	类别标记	得分	查准率	查全率	AUC-PR	AP
0			1.0000	0.0000	-	-
1	1	1.0	1	0.2	0.2*1	0.2*1
2	2	0.9	1/2	0.2	0	0
3	1	0.8	2/3	0.4	0.2*7/12	0.2*2/3
4	1	0.7	3/4	0.6	0.2*17/24	0.2*3/4
5	2	0.6	3/5	0.6	0	0
6	1	0.5	4/6	0.8	0.2*19/30	0.2*4/6
7	2	0.4	4/7	0.8	0	0
8	2	0.3	4/8	0.8	0	0
9	1	0.2	5/9	1	0.2*19/36	0.2*5/9
10	2	0.1	5/10	1	0	0
					1243/1800≈0.691	131/180≈0.728

(b)填表见(a)中表格

AUC-PR和AP都是对PR曲线下方面积的估计，因此二者的值应该相似

(c)交换后，仅最后三行发生改变，列表如下：

下标	类别标记	得分	查准率	查全率	AUC-PR	AP
9	2	0.2	4/9	0.8	0	0
10	1	0.1	5/10	1	0.2*17/36	0.2*5/10
					1223/1800≈0.679	43/60≈0.717

(d)编程结果如下图

```
>>> sum(AUC_PR)
0.6905555555555555
>>> sum(AP)
0.7277777777777777
>>> AUC_PR
[0.2, 0.0, 0.11666666666666665, 0.1416666666666666, 0.0, 0.1266666666666667, 0.0, 0.0, 0.10555555555555554, 0.0]
>>> AP
[0.2, 0.0, 0.13333333333333333, 0.14999999999999997, 0.0, 0.13333333333333336, 0.0, 0.0, 0.11111111111111109, 0.0]
>>>
```

确实与(a)中表格结果相吻合

3、第四章习题8

(a)出现了4次，运行截图如下：

```
Type "help", "copyright", "credits" or "license(
>>> lst=[1,1,2,2,2,2,2,2,2,2]
>>> import random
>>> random.randint(0,10)
5
>>> idx=[]
>>> for i in range(0,10):
        idx=random.sample(range(0,10),5)
        lst1=[]
        for j in idx:
                lst1.append(lst[j])
        print(lst1.count(1))


1
1
0
2
1
0
2
1
1
1
>>>
```

(b)若认为划分后两个子集没有区别，则$(0,2)$出现的概率是$\frac{C_2^2*C_8^3}{C_{10}^5}=2/9$

若认为划分后子集有区别(如测试集和训练集)，则出现概率是$\frac{C_2^2C_8^3+C_8^5}{C_{10}^5}=4/9$

(c)
第一轮中训练集包含2个第1类样本，测试集中全是第2类样本。这种情况对验证没有影响。

第二轮中训练集包含全是第2类样本，测试集中包含2个第1类样本。这种情况下验证集所有样本都会被预测为第2类。

(d)采样的大小为5，总样本数为10，故样本比例为$5/10=0.5$，即每类采样时均按照这个比例确定该类的样本数，对于第1类，其采样时的样本数即为$0.5*2=1$，故采用分层采样时第一类样本的分布始终为$(1,1)$

4、第五章习题1

(a)$XX^T=(U\Sigma V^T)(U\Sigma V^T)^T=U\Sigma V^TV\Sigma^TU^T=U\Sigma\Sigma^TU^T=U\Sigma\Sigma^TU^{-1}$
故XX^T的特征值即为$\Sigma\Sigma^T$的对角线元素，令$k=\min\{m,n\}$，则特征值为$\sigma_1^2,\dots\sigma_k^2$和$m-k$个$0$。特征向量为$U$的对应列向量

(b)与(a)类似，$X^TX = V\Sigma^T\Sigma V^{-1}$
其特征值为$\sigma_1^2, \ldots \sigma_k^2$和$n-k$个$0$。
特征向量为V的对应列向量

(c)他们有k个特征值相同，均为$\sigma_1^2, \ldots, \sigma_k^2$

(d)X的k个奇异值的平方就是$XX^T(X^TX)$的特征值中的k个

(e)XX^T仅为$2*2$的矩阵，故先计算其特征值，X^TX的特征值即为这两个特征值和99998个0

5、第五章习题2

(a)忘记减去平均向量时，第一个特征向量和平均向量之间的corr小于1，且绝对值也较小。
(b)scale变化时，$e1$发生变化，new_e1不变。
正确的特征向量是
$(0.1799, 0.1593, 0.6602, -0.5220, 0.0266, -0.1005, -0.4354, 0.1580, -0.0459, -0.0803)$